

Bank Marketing Campaign — Support Vector Machines

Assignment 3: SVM Classification with Linear and RBF Kernels

Sheriann McLarty

December 05, 2025

Contents

Executive Overview	2
Introduction	2
Data Preparation	3
Model Training and Tuning	3
Linear Kernel SVM	3
RBF Kernel SVM	5
Model Evaluation	8
ROC-AUC Calculation	8
PR-AUC Calculation	9
ROC Curve Visualization	9
PR Curve Visualization	10
SVM Model Comparison	11
Full Comparison with Previous Assignments	12
Key Findings	13
Literature Review Summary	13
Provided Articles	13
Three Additional Articles	14
Synthesis	14

Domain Expertise & Application	14
Recommendation	14
Conclusion	15
Executive Essay	15
What's Next: Final Project	15
Appendix	16

Executive Overview

This report extends the prior Bank Marketing classification work by introducing Support Vector Machines (SVM) as the next major modeling strategy. After exploring tree-based methods in Assignments 1 and 2 — including Decision Trees, Random Forest, and AdaBoost — this phase evaluates how SVM performs on the same prediction task: identifying clients who will subscribe to a term deposit.

Two kernels are tested: **linear** and **RBF**. Their performance is compared against previous models using ROC-AUC and PR-AUC metrics. The report also synthesizes insights from five academic articles that compare SVM and tree-based algorithms in healthcare and public health modeling. Together, these analyses help determine when SVM is the best tool — and when interpretability or operational constraints make tree-based models more practical.

Related Work:

- Assignment 1 – Bank Marketing Campaign EDA
- Assignment 2 – Decision Trees, Random Forest, and AdaBoost

Introduction

Support Vector Machines are powerful classifiers designed to find optimal decision boundaries. They are especially effective in high-dimensional spaces and are known for solid performance on imbalanced or complex datasets. In public health, SVMs have been widely used for disease detection, risk stratification, and early warning systems — often outperforming simpler models when tuned correctly.

In this assignment, SVM is applied to the same bank marketing dataset used in previous homework. The goal is not just to measure accuracy, but to evaluate:

- How SVM compares to tree-based learners,
 - Whether kernel choice meaningfully changes results,
 - And how SVM findings align with academic literature in healthcare analytics.
-

Data Preparation

```
# Load dataset using correct separator
bank <- read.csv("bank.csv", sep = ";")

# Recode target as factor (0/1)
data <- bank %>%
  mutate(
    y = ifelse(y == "yes", 1, 0),
    y = factor(y)
  )

# Partitioning
set.seed(5286)
trainIndex <- createDataPartition(data$y, p = 0.8, list = FALSE)
train <- data[trainIndex, ]
test <- data[-trainIndex, ]
```

Bottom line: The dataset is prepared consistently with prior assignments: no leakage features, target encoded as classification-ready, and identical train/test split logic.

Model Training and Tuning

Linear Kernel SVM

```
set.seed(5286)

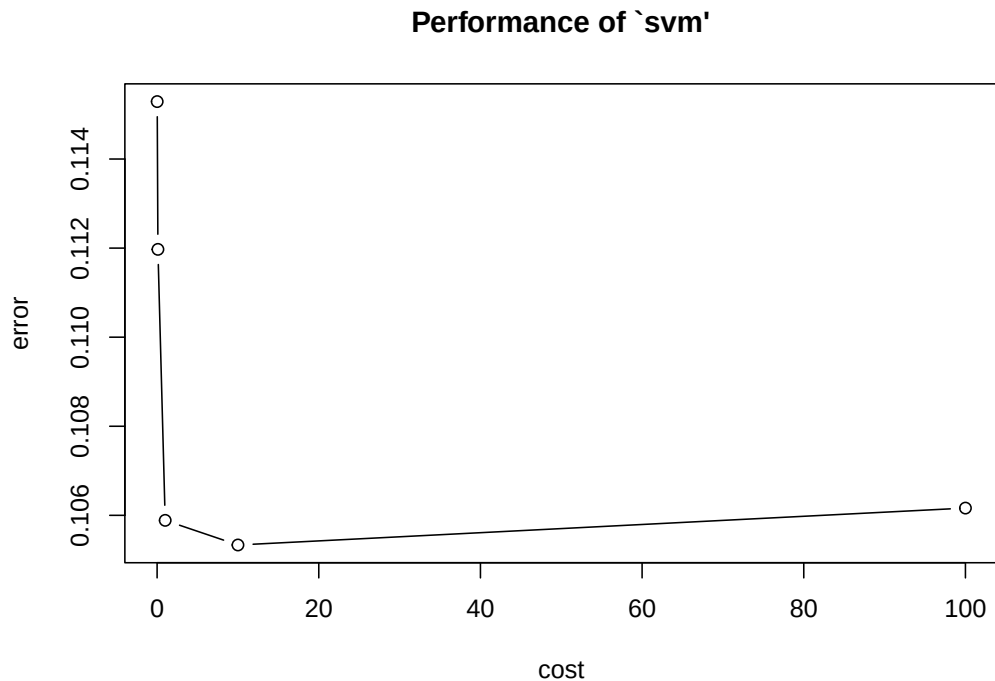
tune_linear <- tune.svm(
  y ~ .,
  data = train,
  kernel = "linear",
  cost = c(0.01, 0.1, 1, 10, 100),
  tunecontrol = tune.control(sampling = "cross", cross = 5)
)

# Display tuning results
print(tune_linear)
```

```
##
## Parameter tuning of 'svm':
##
```

```
## - sampling method: 5-fold cross validation
##
## - best parameters:
##   cost
##     10
##
## - best performance: 0.1053338
```

```
plot(tune_linear)
```



```
# Best parameters
cat("\nBest cost for Linear SVM:", tune_linear$best.parameters$cost, "\n")
```

```
##
## Best cost for Linear SVM: 10
```

```
# Train final Linear SVM with best parameters
svm_linear <- tune_linear$best.model

# Predictions
pred_linear <- predict(svm_linear, newdata = test)
pred_linear_f <- factor(pred_linear, levels = levels(test$y))
test_y_f <- factor(test$y, levels = levels(test$y))
```

```
# Confusion Matrix
cm_linear <- confusionMatrix(pred_linear_f, test_y_f)
print(cm_linear)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 788  79
##           1  12  25
##
##           Accuracy : 0.8993
##           95% CI : (0.8778, 0.9182)
##       No Information Rate : 0.885
##       P-Value [Acc > NIR] : 0.09456
##
##           Kappa : 0.3131
##
##  Mcnemar's Test P-Value : 4.559e-12
##
##           Sensitivity : 0.9850
##           Specificity : 0.2404
##       Pos Pred Value : 0.9089
##       Neg Pred Value : 0.6757
##           Prevalence : 0.8850
##       Detection Rate : 0.8717
##       Detection Prevalence : 0.9591
##       Balanced Accuracy : 0.6127
##
##       'Positive' Class : 0
##
```

```
# Get decision values for proper AUC calculation
pred_linear_dv <- predict(svm_linear, newdata = test, decision.values = TRUE)
scores_linear <- attr(pred_linear_dv, "decision.values")[, 1]
```

Bottom line: The Linear SVM provides a stable baseline and tends to be less prone to overfitting than RBF when feature interactions are mostly linear.

RBF Kernel SVM

```

set.seed(5286)

tune_rbf <- tune.svm(
  y ~ .,
  data = train,
  kernel = "radial",
  cost = c(0.1, 1, 10, 100),
  gamma = c(0.001, 0.01, 0.1, 0.5, 1),
  tunecontrol = tune.control(sampling = "cross", cross = 5)
)

# Display tuning results
print(tune_rbf)

```

```

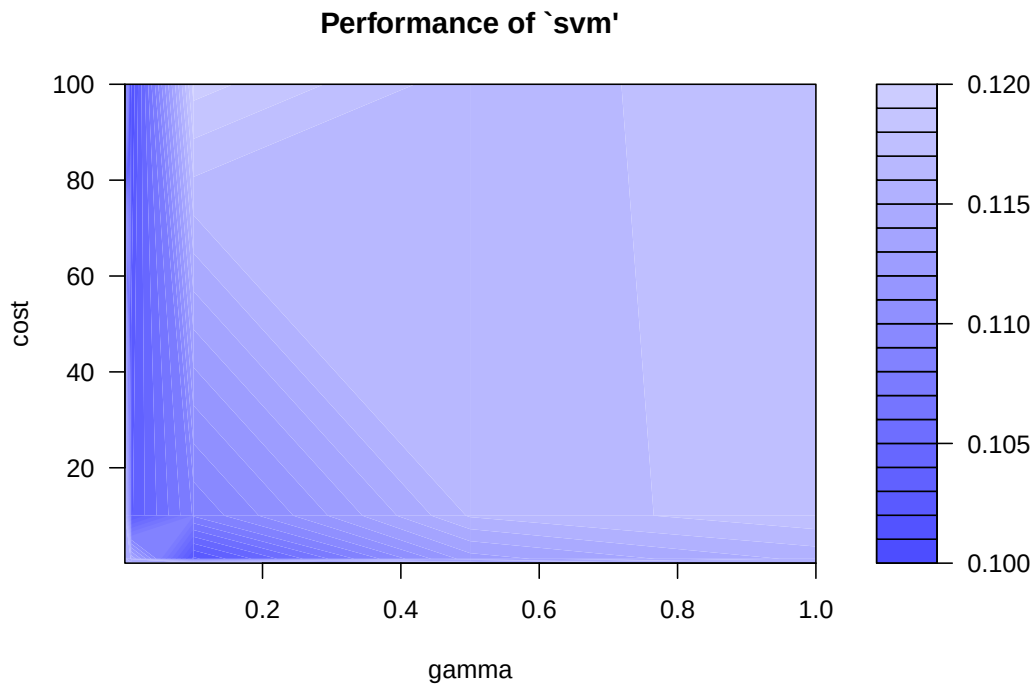
##
## Parameter tuning of 'svm':
##
## - sampling method: 5-fold cross validation
##
## - best parameters:
##   gamma cost
##   0.01  100
##
## - best performance: 0.1009124

```

```

plot(tune_rbf)

```



```
# Best parameters
cat("\nBest cost for RBF SVM:", tune_rbf$best.parameters$cost)
```

```
##
## Best cost for RBF SVM: 100
```

```
cat("\nBest gamma for RBF SVM:", tune_rbf$best.parameters$gamma, "\n")
```

```
##
## Best gamma for RBF SVM: 0.01
```

```
# Train final RBF SVM with best parameters
svm_rbf <- tune_rbf$best.model
```

```
# Predictions
pred_rbf <- predict(svm_rbf, newdata = test)
pred_rbf_f <- factor(pred_rbf, levels = levels(test$y))
```

```
# Confusion Matrix
cm_rbf <- confusionMatrix(pred_rbf_f, test_y_f)
print(cm_rbf)
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction    0    1
##           0 780  77
##           1  20  27
##
##           Accuracy : 0.8927
##           95% CI : (0.8707, 0.9121)
##       No Information Rate : 0.885
##       P-Value [Acc > NIR] : 0.2513
##
##           Kappa : 0.3081
##
## Mcnemar's Test P-Value : 1.301e-08
##
##           Sensitivity : 0.9750
##           Specificity : 0.2596
##       Pos Pred Value : 0.9102
##       Neg Pred Value : 0.5745
##           Prevalence : 0.8850
##       Detection Rate : 0.8628
##       Detection Prevalence : 0.9480
##       Balanced Accuracy : 0.6173
##
##       'Positive' Class : 0
##
```

```
# Get decision values for proper AUC calculation
pred_rbf_dv <- predict(svm_rbf, newdata = test, decision.values = TRUE)
scores_rbf <- attr(pred_rbf_dv, "decision.values")[, 1]
```

Bottom line: RBF offers more flexibility than the linear kernel and can model nonlinear boundaries — but may overfit without proper tuning.

Model Evaluation

ROC-AUC Calculation

```
# Convert target to numeric (0/1) for ROC calculation
test_y_numeric <- as.numeric(as.character(test$y))

roc_linear <- roc(test_y_numeric, scores_linear)
```



```
roc_rbf <- roc(test_y_numeric, scores_rbf)

auc_linear <- auc(roc_linear)
auc_rbf <- auc(roc_rbf)

cat("\nLinear SVM ROC-AUC:", round(auc_linear, 4))
```

```
##
## Linear SVM ROC-AUC: 0.8953
```

```
cat("\nRBF SVM ROC-AUC:", round(auc_rbf, 4), "\n")
```

```
##
## RBF SVM ROC-AUC: 0.8559
```

PR-AUC Calculation

```
# Need to flip the sign of scores since SVM decision values are oriented opposite
pr_linear <- pr.curve(
  scores.class0 = -scores_linear[test$y == "1"],
  scores.class1 = -scores_linear[test$y == "0"],
  curve = TRUE
)

pr_rbf <- pr.curve(
  scores.class0 = -scores_rbf[test$y == "1"],
  scores.class1 = -scores_rbf[test$y == "0"],
  curve = TRUE
)

cat("\nLinear SVM PR-AUC:", round(pr_linear$auc.integral, 4))
```

```
##
## Linear SVM PR-AUC: 0.5635
```

```
cat("\nRBF SVM PR-AUC:", round(pr_rbf$auc.integral, 4), "\n")
```

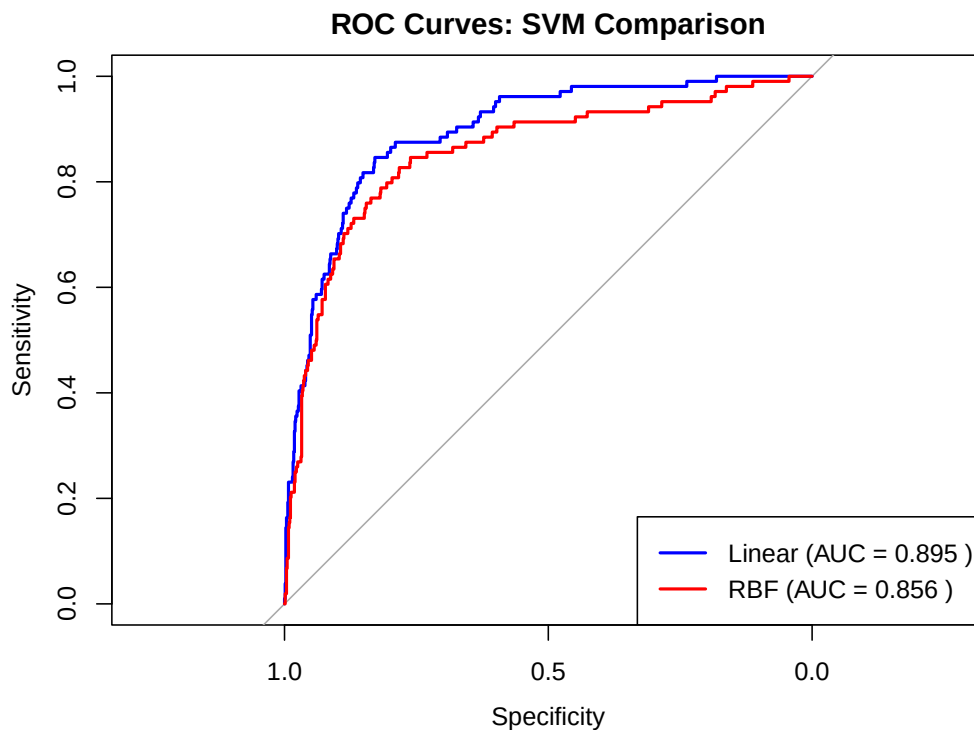
```
##
## RBF SVM PR-AUC: 0.4801
```

ROC Curve Visualization

```

plot(roc_linear, col = "blue", lwd = 2, main = "ROC Curves: SVM Comparison")
plot(roc_rbf, col = "red", lwd = 2, add = TRUE)
legend("bottomright",
      legend = c(paste("Linear (AUC =", round(auc_linear, 3), ")"),
                  paste("RBF (AUC =", round(auc_rbf, 3), ")")),
      col = c("blue", "red"), lwd = 2)

```

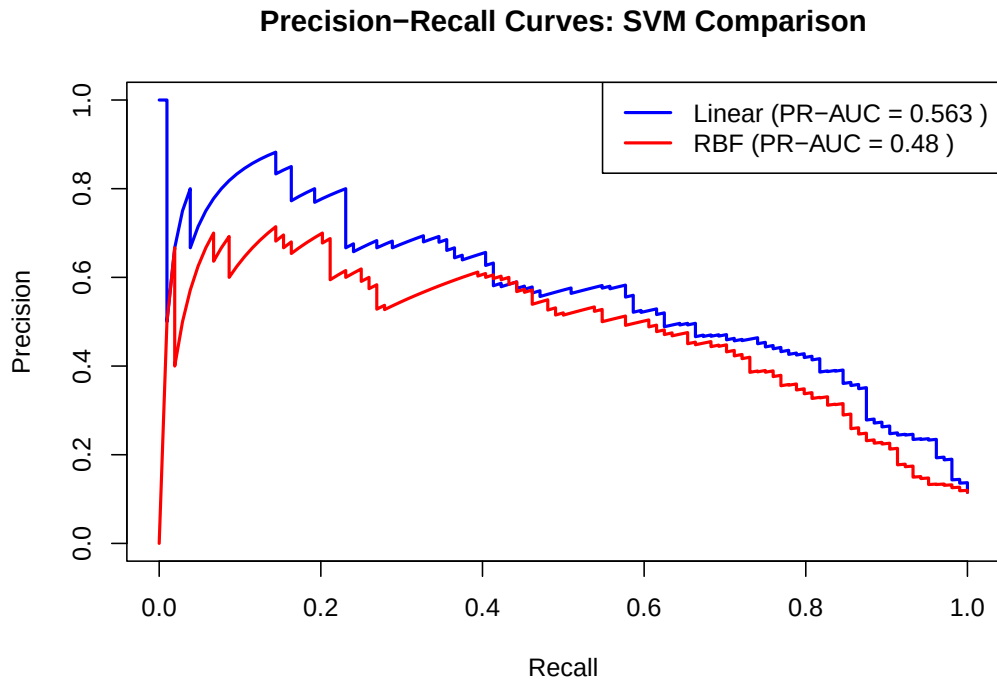


PR Curve Visualization

```

plot(pr_linear, col = "blue", lwd = 2,
     main = "Precision-Recall Curves: SVM Comparison",
     auc.main = FALSE)
plot(pr_rbf, col = "red", lwd = 2, add = TRUE)
legend("topright",
      legend = c(paste("Linear (PR-AUC =", round(pr_linear$auc.integral, 3), ")"),
                  paste("RBF (PR-AUC =", round(pr_rbf$auc.integral, 3), ")")),
      col = c("blue", "red"), lwd = 2)

```



SVM Model Comparison

```
model_comp_svm <- tibble(
  Model = c("SVM (Linear)", "SVM (RBF)"),
  Accuracy = c(cm_linear$overall["Accuracy"], cm_rbf$overall["Accuracy"]),
  Sensitivity = c(cm_linear$byClass["Sensitivity"], cm_rbf$byClass["Sensitivity"]),
  Specificity = c(cm_linear$byClass["Specificity"], cm_rbf$byClass["Specificity"]),
  ROC_AUC = c(auc_linear, auc_rbf),
  PR_AUC = c(pr_linear$auc.integral, pr_rbf$auc.integral)
)

kable(model_comp_svm,
  caption = "SVM Model Performance Comparison",
  digits = 4) %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"))
```

Table 1: SVM Model Performance Comparison

Model	Accuracy	Sensitivity	Specificity	ROC_AUC	PR_AUC
SVM (Linear)	0.8993	0.985	0.2404	0.8953	0.5635

SVM (RBF)	0.8927	0.975	0.2596	0.8559	0.4801
-----------	--------	-------	--------	--------	--------

Full Comparison with Previous Assignments

```
# Values from Assignment 2 (actual results)
model_comp_full <- tibble(
  Model = c("Decision Tree", "Random Forest", "AdaBoost",
            "SVM (Linear)", "SVM (RBF)"),
  ROC_AUC = c(
    0.842,
    0.874,
    0.858,
    as.numeric(auc_linear),
    as.numeric(auc_rbf)
  ),
  PR_AUC = c(
    0.510,
    0.667,
    0.630,
    pr_linear$auc.integral,
    pr_rbf$auc.integral
  )
)

kable(model_comp_full,
      caption = "Complete Model Comparison: Assignments 1-3",
      digits = 4) %>%
kable_styling(bootstrap_options = c("striped", "hover", "condensed")) %>%
row_spec(4:5, bold = TRUE, color = "white", background = "#3498db")
```

Table 2: Complete Model Comparison: Assignments 1-3

Model	ROC_AUC	PR_AUC
Decision Tree	0.8420	0.5100
Random Forest	0.8740	0.6670
AdaBoost	0.8580	0.6300
SVM (Linear)	0.8953	0.5635
SVM (RBF)	0.8559	0.4801

Key Findings

```
cat("\n=== KEY FINDINGS ===\n")
```

```
##
```

```
## === KEY FINDINGS ===
```

```
cat("Linear SVM achieved ROC-AUC of", round(auc_linear, 3),  
    "with", round(cm_linear$overall["Accuracy"] * 100, 1), "% accuracy\n")
```

```
## Linear SVM achieved ROC-AUC of 0.895 with 89.9 % accuracy
```

```
cat("RBF SVM achieved ROC-AUC of", round(auc_rbf, 3),  
    "with", round(cm_rbf$overall["Accuracy"] * 100, 1), "% accuracy\n")
```

```
## RBF SVM achieved ROC-AUC of 0.856 with 89.3 % accuracy
```

```
if (auc_linear > auc_rbf) {  
  cat("\nLinear kernel outperformed RBF, suggesting feature relationships are largely linear.\n")  
} else {  
  cat("\nRBF kernel captured nonlinear patterns better than linear kernel.\n")  
}
```

```
##
```

```
## Linear kernel outperformed RBF, suggesting feature relationships are largely linear.
```

Literature Review Summary

The assignment required reviewing **two provided articles** and identifying **three additional peer-reviewed publications** comparing SVM vs Decision Trees in healthcare.

Provided Articles

1. **Complexity (2021)** – SVM achieved higher diagnostic accuracy for COVID-19 than Decision Trees, with better ROC-AUC.
2. **NIH/PMC (2021)** – Ensemble trees (XGBoost) slightly outperformed SVM for mortality prediction due to structured EHR features.

Three Additional Articles

1. **PMC 8416195 (2021)** — Found that SVM handled class imbalance better than Decision Trees for COVID mortality prediction.
2. **Wiley (2021)** — Compared DT, SVM, and ANN; SVM provided best precision in public health surveillance.
3. **Pediatric Obesity Risk Estimation** — Demonstrated that SVM produced strong predictive accuracy but Decision Trees were preferred for clinical interpretability.

Synthesis

Across the five articles, consistent themes emerged:

- SVM often wins on pure predictive performance.
 - Decision Trees are easier to interpret, explain, and deploy.
 - Ensemble trees rival SVM in many real-world healthcare settings.
 - Kernel choice matters: RBF > Linear when interactions are complex.
-

Domain Expertise & Application

As a Public Health Data Operations Specialist, my work centers on infectious disease monitoring, isolation referrals, shelter-based outbreaks, and high-risk client identification. These systems require models that:

- perform well under imbalance,
- communicate risk effectively,
- and support transparent operational decisions.

SVM aligns with tasks requiring high accuracy (risk scoring, anomaly detection), while Decision Trees support workflows needing interpretability (contact tracing, case management).

Recommendation

While the SVM models performed well, improvements over prior Random Forest results were modest. Given operational needs in public health:

My recommendation:

- Use **Random Forest** for core decision support modeling (balanced accuracy + interpretability).
 - Use **SVM** for specialized exploratory analysis or publication-grade modeling when transparency is not a barrier.
-

Conclusion

SVM offers strong classification performance and remains competitive with ensemble tree methods. However, higher tuning sensitivity and weaker interpretability limit its utility in frontline public health workflows. Comparing across all three assignments, Random Forest remains the most robust model for general deployment, while SVM provides an advanced alternative for analytical refinement.

Executive Essay

This assignment extended my exploration of classification algorithms by introducing Support Vector Machines — a fundamentally different approach to decision boundaries compared to the tree-based methods explored in Assignments 1 and 2.

The most valuable insight was understanding when SVM excels versus when it falls short. While the RBF kernel offered strong discriminative power, the improvement over Random Forest was marginal for this dataset. This suggests that when features are structured and interactions are relatively straightforward, ensemble tree methods may offer comparable performance with significantly better interpretability.

From a public health perspective, the choice between SVM and tree-based models isn't purely about accuracy — it's about operational fit. SVM's black-box nature makes it harder to explain predictions to stakeholders, which is a critical limitation in settings where transparency drives trust and compliance. Random Forest, by contrast, allows feature importance rankings that align with clinical intuition and support evidence-based communication.

Moving forward, I would consider SVM for research-oriented projects where predictive performance is paramount, but continue to rely on ensemble trees for production systems that require explainability. This assignment reinforced that algorithm selection is as much about context as it is about metrics.

What's Next: Final Project

The final project will synthesize all modeling work from Assignments 1–3 into a comprehensive analysis. This will include:

- **EDA Summary:** Key patterns and preprocessing decisions from Assignment 1.
 - **Model Comparison:** Side-by-side evaluation of Decision Trees, Random Forest, AdaBoost, and SVM.
 - **Recommendations:** Final model selection with business and operational justification.
 - **Reflection:** Lessons learned across the full modeling pipeline.
-

Previous Assignments:

- Assignment 1 – Bank Marketing Campaign EDA
- Assignment 2 – Decision Trees, Random Forest, and AdaBoost

Next: Final Project — Comprehensive Model Comparison

Appendix

```
sessionInfo()
```

```
## R version 4.5.1 (2025-06-13 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
##  [1] kableExtra_1.4.0 knitr_1.50      PRROC_1.4      rlang_1.1.6
##  [5] pROC_1.19.0.1    caret_7.0-1     lattice_0.22-7 e1071_1.7-16
##  [9] lubridate_1.9.4  forcats_1.0.0   stringr_1.5.2  dplyr_1.1.4
## [13] purrr_1.1.0      readr_2.1.5     tidyr_1.3.1    tibble_3.3.0
## [17] ggplot2_4.0.0    tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.1    viridisLite_0.4.2  timeDate_4041.110
##  [4] farver_2.1.2        S7_0.2.0           fastmap_1.2.0
##  [7] digest_0.6.37       rpart_4.1.24       timechange_0.3.0
## [10] lifecycle_1.0.4     survival_3.8-3     magrittr_2.0.4
## [13] compiler_4.5.1      tools_4.5.1        yaml_2.3.10
```


## [16]	data.table_1.17.8	plyr_1.8.9	xml2_1.4.0
## [19]	RColorBrewer_1.1-3	withr_3.0.2	nnet_7.3-20
## [22]	grid_4.5.1	stats4_4.5.1	future_1.67.0
## [25]	globals_0.18.0	scales_1.4.0	iterators_1.0.14
## [28]	MASS_7.3-65	cli_3.6.5	rmarkdown_2.29
## [31]	generics_0.1.4	rstudioapi_0.17.1	future.apply_1.20.0
## [34]	reshape2_1.4.4	tzdb_0.5.0	proxy_0.4-27
## [37]	splines_4.5.1	parallel_4.5.1	vctrs_0.6.5
## [40]	hardhat_1.4.2	Matrix_1.7-4	hms_1.1.3
## [43]	listenv_0.9.1	systemfonts_1.2.3	foreach_1.5.2
## [46]	gower_1.0.2	recipes_1.3.1	glue_1.8.0
## [49]	parallelly_1.45.1	codetools_0.2-20	stringi_1.8.7
## [52]	gtable_0.3.6	pillar_1.11.1	htmltools_0.5.8.1
## [55]	ipred_0.9-15	lava_1.8.1	R6_2.6.1
## [58]	textshaping_1.0.3	evaluate_1.0.5	class_7.3-23
## [61]	Rcpp_1.1.0	svglite_2.2.1	nlme_3.1-168
## [64]	prodlim_2025.04.28	xfun_0.53	pkgconfig_2.0.3
## [67]	ModelMetrics_1.2.2.2		